

Laboratory 01

Toolchain & Environment Verification

Real-Time Operating Systems • M.Eng. ECE • KMUTNB

Platform	NXP FRDM-MCXXN236 (ARM Cortex-M33 @ 96 MHz)
RTOS	FreeRTOS (bare-metal, no SDK)
Toolchain	ARM GNU Toolchain (arm-none-eabi-gcc) + Make
Tools	pyocd, SEGGER SystemView
Estimated time	2–3 hours
CLO mapping	CLO 1 (foundations) CLO 3 (FreeRTOS implementation)

1. Objectives

By completing this lab you will be able to:

1. Build and flash a bare-metal FreeRTOS project using the ARM cross-compiler toolchain.
2. Verify that FreeRTOS creates and schedules tasks correctly by reading UART output.
3. Integrate SEGGER SystemView into a FreeRTOS project and capture a real-time trace.
4. Identify task context switches on the SystemView timeline.
5. Observe how task priority affects scheduling — the foundation for Module 2.

2. Prerequisites

Complete the **Lab 1 Setup Guide** (lab01_setup_guide.md) before starting:

- ☐ ARM GNU Toolchain installed; arm-none-eabi-gcc is on PATH (or toolchain path set in Makefile)
- ☐ pyocd installed (pip install pyocd)
- ☐ FRDM-MCXXN236 connected via the **MCU-LINK USB** port
- ☐ MCU-Link firmware **re-flashed to J-Link** (Setup Guide Step 3)
- ☐ SEGGER SystemView installed on your laptop

3. Project Structure

```
lab01_setup_verify/
  Makefile           <- build, flash, and debug targets
  linker_MCXXN236.ld <- flash/RAM memory map
  src/
    startup_MCXXN236.s <- vector table + Reset_Handler
    main.c             <- hw_init, tasks, scheduler start
    FreeRTOSConfig.h   <- kernel configuration
    uart.h / uart.c    <- bare-metal LPUART4 driver
    SystemView/        <- (created in Part C -- you add these)
    FreeRTOS-Kernel/   <- (cloned in Part A)
```

3.1. What the code does

Two FreeRTOS tasks run in this lab:

Task	Priority	Period	What it does
vBlinkTask	2 (HIGH)	500 ms	Prints LED ON / LED OFF plus a toggle counter
vHeartbeatTask	1 (LOW)	1000 ms	Prints the FreeRTOS tick uptime in milliseconds

A shared **UART mutex** (xUartMutex) prevents the two tasks from printing at the same time — the simplest possible example of a critical section.

The key hardware initialisation in `main.c` switches the MCXN236 core clock from the default FRO 12 MHz to FRO 96 MHz before the scheduler starts, so that `configCPU_CLOCK_HZ` matches the actual oscillator.

4. Part A — Build and Flash

4.1. Step 1 — Clone FreeRTOS-Kernel

```
cd labs/lab01_setup_verify
make setup
```

This runs `git clone -depth=1` to fetch the FreeRTOS kernel source into `FreeRTOS-Kernel/`. You only need to do this once.

4.2. Step 2 — Build

```
make
```

Expected output (sizes will vary slightly):

```
arm-none-eabi-gcc ... -o lab01.elf
arm-none-eabi-objcopy -O binary lab01.elf lab01.bin
arm-none-eabi-size lab01.elf
   text    data     bss     dec      hex filename
  12xxx     xxx   xxxxxx  xxxxxx   xxxx lab01.elf
```

Note

If `arm-none-eabi-gcc`: not found, either add the toolchain directory to `PATH`, or set the path at the top of Makefile:

```
TOOLCHAIN ?= /path/to/arm-none-eabi/bin
```

4.3. Step 3 — Flash

Connect the board, then:

```
make flash
```

`pyocd` erases, programs, and resets the MCU. The board starts running immediately.

5. Part B — UART Output Verification

Open a serial terminal on your laptop. The board exposes a CDC-ACM virtual COM port over the same USB cable used for flashing.

macOS / Linux:

```
ls /dev/tty.usbmodem*           # find the port
screen /dev/tty.usbmodemXXXX 115200
# or: minicom -b 115200 -D /dev/tty.usbmodemXXXX
```

Windows: PuTTY or Tera Term — COMx, 115200 baud, 8N1.

5.1. Expected output

```
=== RTOS Lab 01: Toolchain & Environment Verify ===
Tasks: Blink (500 ms) | Heartbeat (1000 ms)
SystemView: disabled (rebuild with: make SYSVIEW=1)

[MAIN] Starting scheduler...

[BLINK] LED ON    (toggle #1)
[BLINK] LED OFF   (toggle #2)
[HB]      uptime = 1000 ms
[BLINK] LED ON    (toggle #3)
[BLINK] LED OFF   (toggle #4)
[HB]      uptime = 2000 ms
```

Tip

Key observations:

- [BLINK] prints *twice* for every one [HB] line, reflecting 500 ms vs. 1000 ms periods.
- Uptime increments by exactly 1000 ms each line — the FreeRTOS 1 kHz tick is running correctly.

Checkpoint

Show a terminal capture of at least 5 seconds of output to your instructor.

6. Part C — SEGGER SystemView Integration

The setup guide (Step 4) covers the SystemView integration in detail. Follow the steps below to add the source files to `src/SystemView/` and rebuild with the `SYSVIEW=1` flag.

6.1. Step 1 — Copy SystemView source files

Locate your SystemView installation. On macOS it is typically:

```
/Applications/SEGGER/SystemView_V3xx/Src/
```

Create the folder `src/SystemView/` and copy:

```
SEGGER_SYSVIEW.c          SEGGER_SYSVIEW.h
SEGGER_SYSVIEW_ConfDefaults.h  SEGGER_SYSVIEW_Int.h
SEGGER_RTT.c              SEGGER_RTT.h
SEGGER_RTT_Conf.h         SEGGER_RTT_printf.c
Config/SEGGER_SYSVIEW_Config_FreeRTOS.c
Sample/OS/SEGGER_SYSVIEW_FreeRTOS.h
```

6.2. Step 2 — Configure the target descriptor

Open `src/SystemView/SEGGER_SYSVIEW_Config_FreeRTOS.c` and update:

```
1 #define SYSVIEW_APP_NAME      "Lab 01 -- Setup Verify"
2 #define SYSVIEW_DEVICE_NAME  "Cortex-M33"
3 #define SYSVIEW_RAM_BASE      0x20000000u
```

6.3. Step 3 — Build with SystemView enabled

```
make sysview          # equivalent to: make SYSVIEW=1
```

The Makefile adds `-DUSE_SYSVIEW` and compiles the SystemView source files automatically. `main.c` calls `SEGGER_SYSVIEW_Conf()` before `vTaskStartScheduler()` when this flag is set.

6.4. Step 4 — Capture a trace

- Flash the new binary:

```
make SYSVIEW=1 flash
```

- Open **SEGGER SystemView** on your laptop.
- Go to **Target** → **Start Recording**.
- Enter connection parameters: **Target Device:** MCMX236 **Interface:** SWD **Speed:** Auto
- Click **OK**.

You should see the real-time Gantt chart with **Blink** and **HB** tasks alternating every 500 ms and 1000 ms.

✓ Checkpoint

Screenshot the SystemView timeline showing both tasks over at least 2 seconds of recording.

7. Part D — Experiments

Work through the experiments in order. For each one, record what you observe (terminal output or SystemView screenshot) and answer the question that follows.

7.1. Experiment 1 — Priority swap

In `main.c`, swap the task priorities:

```
1 xTaskCreate(vBlinkTask,      "Blink", configMINIMAL_STACK_SIZE, NULL,
    1, NULL); /* was 2 */
2 xTaskCreate(vHeartbeatTask, "HB",      configMINIMAL_STACK_SIZE, NULL,
    2, NULL); /* was 1 */
```

Rebuild, flash, and observe.

? Question

Does the output order or timing change? Why or why not?

Hint: both tasks spend almost all their time blocked inside `vTaskDelay`. Think about what happens when both unblock at the exact same tick.

7.2. Experiment 2 — Equal priorities

Set both tasks to the same priority:

```
1 xTaskCreate(vBlinkTask,      "Blink", configMINIMAL_STACK_SIZE, NULL,
    1, NULL);
2 xTaskCreate(vHeartbeatTask, "HB",      configMINIMAL_STACK_SIZE, NULL,
    1, NULL);
```

? Question

What scheduling policy does FreeRTOS apply when two tasks have equal priority and are both ready? Where exactly in `FreeRTOSConfig.h` is this behaviour configured?

7.3. Experiment 3 — Periods and CPU load

Restore the original priorities (`Blink = 2`, `HB = 1`). Change `vBlinkTask`'s period from 500 ms to 50 ms:

```
1 vTaskDelay(pdMS_TO_TICKS(50));
```

Rebuild and capture a SystemView trace.

? Question

Open the **CPU Load** view in SystemView (**View** → **CPU Load**). What fraction of CPU time does **Blink** consume?

Now compute the utilization by hand using:

$$U = \frac{C_{\text{Blink}}}{T_{\text{Blink}}} + \frac{C_{\text{HB}}}{T_{\text{HB}}}$$

Estimate C_i from the trace. Do the measured and computed values agree? Explain any discrepancy.

7.4. Experiment 4 — Remove the mutex (observe the problem)

Keep the 50 ms Blink period from Experiment 3. In both task bodies, comment out the `xSemaphoreTake` / `xSemaphoreGive` calls (keep the `uart_printf` lines):

```
1 /* xSemaphoreTake(xUartMutex, portMAX_DELAY); */
2 uart_printf("[BLINK] ... \n");
3 /* xSemaphoreGive(xUartMutex); */
```

? Question

Do you observe corrupted or interleaved output on the terminal?

Why is this a **race condition** even on a *single-core* microcontroller? What category of kernel primitive fixes it, and exactly what resource does the mutex protect here?

! Note

Restore the mutex calls before submitting your work.

8. Deliverables

Submit the following to the course LMS by the posted deadline:

#	Item	Details
1	Terminal capture	5+ seconds showing correct [BLINK] / [HB] output
2	SystemView screenshot	Timeline with both tasks, ≥ 2 seconds of recording
3	Answers to Q1–Q4	A few sentences each; reference the code or trace
4	Modified <code>main.c</code>	From Experiment 3 (50 ms Blink, mutex restored)

9. Key Concepts Introduced

Concept	Where you saw it
Preemptive, priority-based scheduling	Priority swap (Experiment 1)
Time-slicing among equal-priority tasks	Equal priority (Experiment 2)
CPU utilization $U = C/T$	CPU Load view (Experiment 3)
Race condition on a shared resource	Mutex removal (Experiment 4)
Mutex as a critical-section guard	<code>xUartMutex</code> throughout
Periodic task with <code>vTaskDelay</code>	<code>vBlinkTask</code> , <code>vHeartbeatTask</code>

These are the building blocks for Module 2 (scheduling theory) and Module 3 (synchronisation).

10. Reference

Resource	Relevant sections
Barry, <i>Mastering the FreeRTOS Kernel</i>	Ch. 1 (tasks), Ch. 3 (queue/semaphore basics)
Laplace, <i>Real-Time Systems Design</i>	Ch. 1–2 (task model, deadline classes)
FreeRTOS documentation	<code>xTaskCreate</code> , <code>vTaskDelay</code> , <code>xSemaphoreCreateMutex</code>
SEGGER SystemView User Guide	Recording setup, CPU Load view
